Nombre de la Prueba (Aserción)	Entrada de datos	Salida Esperada (Resultados)
assertEquals CalcularSalarioBruto	new EmpleadoVentas("Juan", "123", 30, 1000000f, JUNIOR, 4f, 4f, 1000000f, 10f)	1150000
assertNotEquals SalarioNeto Temporal	new EmpleadoTemporal("Ana", "456", 25, 1500f, SEMI_SENIOR, 4f, 4f, 20, 80f)	salario neto ≠ 0
assertIterableEquals Empleados almacenados	Agregar 3 empleados en orden: e1, e2, e3	La lista devuelve [e1, e2, e3] en el mismo orden de registro
assertTrue Bonificación JUNIOR > 0	new EmpleadoPlanta(..., JUNIOR, salarioBase = 1000000f, ...)	calcularBonificacionCategoria() > 0
assertTrue Salario neto nunca negativo	Cualquier empleado válido (planta, temporal o ventas)	calcularSalarioNeto() >= 0
assertNull Búsqueda empleado inexistente	empresa.buscarEmpleadoPorDocumento("999999")	null
assertThrows Salario base negativo	new EmpleadoVentas(..., salarioBase = -100000f, ...)	Lanza IllegalArgumentException
assertEquals empleadosConSalarioMayorA()	Empresa con empleados de salario neto: 800, 1500, 1200. Llamar empleadosConSalarioMayorA(1000)	Lista con [empleado1500, empleado1200] (mismo orden)
assertNull buscarPorDocumento inexistente	empresa.buscarEmpleadoPorDocumento("999999")	null
assertThrows No permitir duplicados	Agregar empleado con documento "123", luego intentar agregar otro con el mismo documento	Lanza IllegalArgumentException
assertEquals obtenerEmpleadoConMayorSalarioNeto()	Empresa con salarios neto: 800, 2200, 1500	Retorna el empleado con 2200
assertEquals Empleados temporales > 100 días	2 empleados temporales (50 días y 150 días) → llamar método de temporales > 100	Lista con solo el de 150 días
assertTrue SalarioNeto Planta > Salario Base	new EmpleadoPlanta(salarioBase=2000000, horasExtra=10, valorHoraExtra=50000, auxilio=180000)	salarioNeto > salarioBase